Balloon Pop Game Project

Week 3 Progress Report

Project Name: Balloon Pop Game

Development Tool: Apache NetBeans IDE 28

Programming Language: Java

GUI Library: Java Swing

Objective

The objective of Week 3 was to add the first balloon to the Balloon Pop Game and implement its basic movement. The balloon was designed to move upward continuously and return to the bottom at a different random horizontal position after reaching the upper area of the game window.

Work Completed

The following tasks were successfully completed during Week 3:

- Added the first balloon to the existing game window.
- Created the balloon using Java's "Graphics" class.
- Added a red balloon with a black string.
- Added a Start button action so that the game begins when the Start button is clicked.

- Added a Swing "Timer" to control the balloon's movement.

- Implemented upward movement by continuously changing the balloon's Y-coordinate.

- Added a random position system using Java's "Random" class.

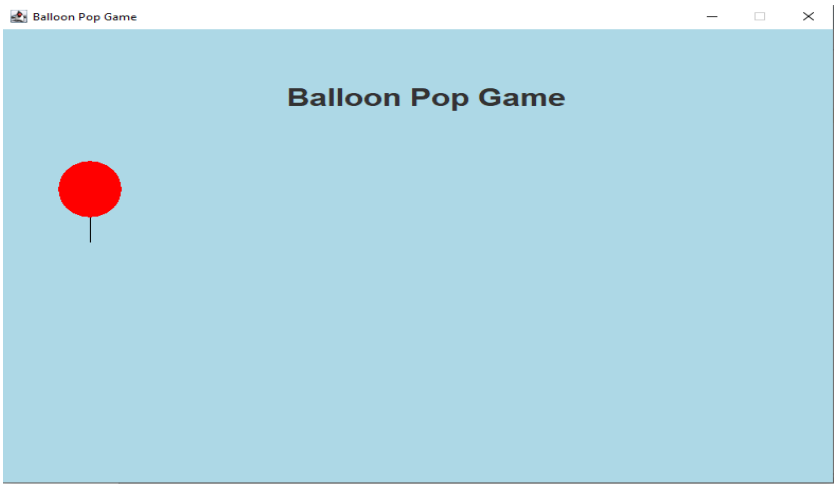
- When the balloon reaches the upper area, it returns to the bottom.

- The balloon receives a new random X-coordinate when it returns to the bottom.

- Used "repaint()" to refresh the window and display the balloon's updated position.

- Tested the program successfully in Apache NetBeans IDE 28.

Screenshot of Week 3 GUI



Output

1.After clicking the Start button, the Start button disappears and a red balloon appears near the bottom of the game window.

2.The balloon continuously moves upward. When it reaches the upper limit, it returns to the bottom and appears at a new random horizontal position.

3.This creates basic balloon movement and makes the game more dynamic.

Program Logic

The basic logic of the Week 3 implementation is:

1. The game window is created.

2. The user clicks the Start button.

3. The "startGame()" method is called.

4. The balloon is placed at its starting position.

5. The movement timer starts.

6. The "moveBalloon()" method is repeatedly called by the timer.

7. The balloon's Y-coordinate is decreased, causing it to move upward.

8. When the balloon reaches the upper area, its Y-coordinate is reset to the bottom.

9. A new random X-coordinate is generated.

10. "repaint()" redraws the balloon at its new position.

11. The process continues while the game is running.

Challenges Faced

During Week 3, the main challenges were:

- Understanding how to draw a balloon using the "Graphics" class.
- Understanding X and Y coordinates for positioning the balloon.
- Understanding how a Swing "Timer" creates continuous movement.
- Understanding how changing the Y-coordinate moves the balloon upward.
- Learning how to generate a random horizontal position using the "Random" class.
- Understanding the purpose of "repaint()" when the balloon's position changes.

These challenges were solved through practice, testing, and debugging.

Learning Outcomes

After completing Week 3, I learned:

- How to draw basic shapes using Java "Graphics".
- How to control the position of a game object using X and Y coordinates.
- How to use a Swing "Timer" for repeated actions.
- How to create basic animation by repeatedly changing an object's position.
- How to use the "Random" class to generate different positions.

- How "repaint()" refreshes the graphical display.

- How to connect a button with a game action using an action listener.

Code Logic Explanation

Code| Logic / Purpose

1."import javax.swing.Timer;"| Imports the Swing Timer used to repeatedly update the balloon's position.

2."import java.awt.Graphics;"| Imports the Graphics class used to draw the balloon and its string.

3."import java.util.Random;"| Imports the Random class to generate a new horizontal balloon position.

4."private int balloonX = 370;"| Stores the balloon's horizontal position.

5."private int balloonY = 500;"| Stores the balloon's vertical starting position.

6."private Timer balloonTimer;"| Declares the timer responsible for repeated balloon movement.

7."private Random random = new Random();"| Creates a Random object for generating random positions.

8."private boolean gameStarted = false;"| Keeps track of whether the game has started.

9.startButton.addActionListener(e -> startGame());"| Calls "startGame()" when the Start button is clicked.

10."balloonTimer = new Timer(30, e -> moveBalloon());"| Calls "moveBalloon()" repeatedly at approximately 30-millisecond intervals.

11."gameStarted = true;"| Changes the game state to started.

12."startButton.setVisible(false);"| Hides the Start button after the game begins.

13."balloonY = balloonY - 3;"| Decreases the Y-coordinate, moving the balloon upward.

14."if (balloonY < 100)"| Checks whether the balloon has reached the upper area.

15."balloonY = 500;"| Sends the balloon back to the bottom.

16."balloonX = random.nextInt(700) + 20;"| Gives the balloon a new random horizontal position.

17."repaint();"| Requests the window to redraw the balloon at its updated position.

18."g.setColor(Color.RED);"| Sets the drawing color to red.

19."g.fillOval(balloonX, balloonY, 60, 70);"| Draws the red balloon as a filled oval.

20."g.setColor(Color.BLACK);"| Changes the drawing color to black for the string.

21."g.drawLine(...)"| Draws the balloon's string.

Conclusion

Week 3 was completed successfully. The first moving balloon was added to the existing Balloon Pop Game GUI. The balloon can now move upward continuously and return from the bottom at a different random horizontal position. This provides the basic animation and movement foundation required for developing multiple balloons and more advanced gameplay features in Week 4.